

Home

- [0.0] Prerequisites
- [0.1] Ray Tracing
- [0.2] CNNs & ResNets
- [0.3] Optimization
- [0.4] Backprop
- [0.5] VAEs & GANs

Table of Contents

AI Safety

Why ARENA?

How to access the course

Option 1: Colab

Option 2: VS Code (blank file)

Option 3: VS Code (notebooks)

ARENA structure

Structure of exercises

VS Code setup

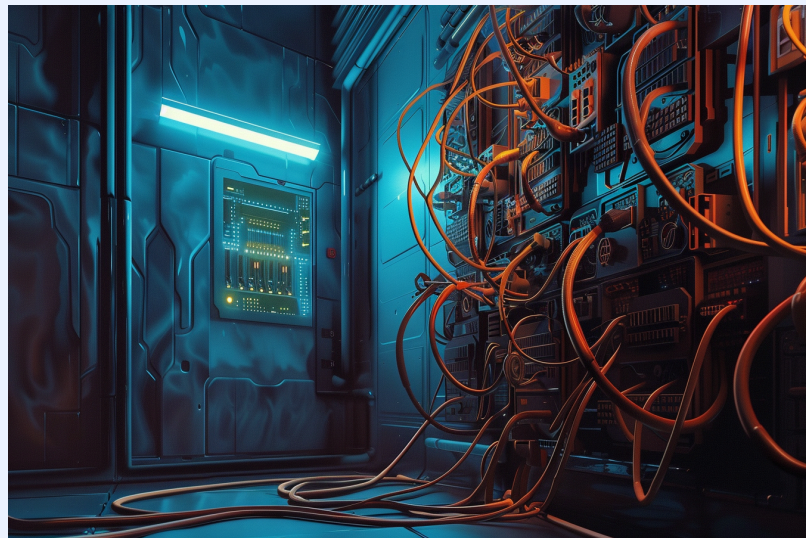
System Requirements

Local setup instructions

VM setup instructions

Set up your workspace on your VM

Feedback



Chapter 0: Fundamentals

Links to all other ARENA chapters:

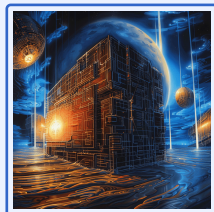
- [Chapter 0: Fundamentals](#)
- [Chapter 1: Transformer Interpretability](#)
- [Chapter 2: Reinforcement Learning](#)
- [Chapter 3: LLM Evaluations](#)

The material on this page covers the first five days of the curriculum. It can be seen as a grounding in all the advanced sections of this course (such as RL, transformers, mechanistic interpretability, and generative models).

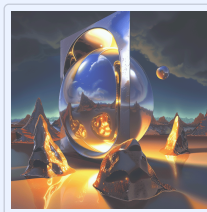
Some highlights from this chapter include:

- Building your own 1D and 2D convolution functions
- Building and loading weights into a Residual Neural Network, and finetuning it on a classification task
- Working with [weights and biases](#) to optimise hyperparameters
- Implementing your own backpropagation mechanism
- Building your own VAEs & GANs, and using them to generate images

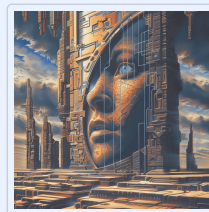
Click to see a summary of each page (use the left hand sidebar to actually visit the pages):



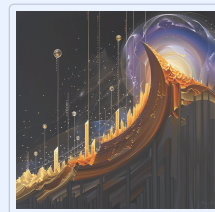
Prerequisites



Raytracing



CNNs & ResNets



Optimization &
Hyperparameters



Backprop

AI Safety

As AIs become more powerful and are deployed autonomously in high-stakes contexts, it will be increasingly important to prevent them from causing harmful outcomes. We need to figure out how to build tools that increase our defence against the amazing upsides brought by AI progress. In order to do this, there is a collection of highly consequential and technical research areas, and the aim of this course is to equip you with the ML engineering skills to tackle them.

If you aren't very familiar with AI safety, we recommend watching this video and reading some of the resources.



- [Research by DeepMind](#) has shown examples of real-world reinforcement learning agents demonstrating they elucidate specification gaming, one class of phenomena showing how difficult getting an AI's actions to match this can result in strange and unexpected failures.
- [This post by Paul Christiano](#) sketches out a possible scenario of how AI development could go wrong over the long problem. While this is only a story, it is very useful for thinking about how AI take-off could look in the real world.
- [This paper by Hendrycks et al.](#), which gives a clear and broad overview of the technical problems in AI safety.
- [This paper by Ngo et al.](#) gives an overview of how the alignment problem interacts with the modern AI problem.
- [This course by BlueDot Impact](#) provides resources on a multitude of topics in technical AI safety, with a lot of blogposts, so that you can gain an overview of the field as it exists today.

Why ARENA?

My first proper exposure to technical AI safety content was at the [MLAB workshop run by Redwood Research](#). I found the material felt different to previous AI safety work I'd read about and engaged in. I'd participated in some of it and tried to get involved in the work being done, but if I could summarize the difference — MLAB felt like I was being asked to rather than just impersonating what I thought an AI safety technical researcher would be doing. Unfortunately, this is a pre-paradigmatic field where there are so many different approaches and perspectives that can be brought to bear on the effectiveness of your research directions. But with certain areas of safety research entering the mainstream funding/interest/literature, programs like MLAB which aim to provide a concrete guided introduction to the field are becoming more common.

This is where ARENA came in. It was designed partly to be a spiritual successor to MLAB, but with the ability to focus on technical AI safety like interpretability, RLHF, and evals. The first iteration was run with just five participants, but since then has grown into a large community of people who are passionate about AI safety research, and programs like [MATS](#) and [AI safety camp](#), as well as full-time roles at companies like [Anthropic](#) and [Conjecture](#).

ARENA certainly couldn't exist without the wealth of fantastic open-source material that has been published over the years, from [transformer circuits thread](#), to Neel Nanda's [TransformerLens library](#), to the original MLAB material, to many others. Around it, it can be hard to know what to focus on or prioritize. I see the purpose of ARENA largely as structuring and organizing this material, for anyone who finds themselves overwhelmed by the amount of technical AI safety content out there. As the material in and the community around ARENA continues to grow over the coming months and years, and though it may not have specific domain knowledge, it can serve as a springboard for their career development in technical AI safety.

— [Callum McDougall](#), ARENA Cofounder

How to access the course

There are 3 main ways to access the course:

Option 1: Colab

For each day of exercises, there are 2 Colab notebooks: an exercises & solutions notebook. You can make a copy of each by one, filling in the code cells as indicated. Solutions are found in dropdowns below the code cells in the exercises notebook. The exercises notebook also has all inline output displayed.

We recommend this option if you're more familiar with Colab / want to avoid the hassle of setting up your own environment.

► Click this dropdown for links to each of the colab exercises.

Importantly, **this option doesn't rely on the Streamlit pages at all** - the relevant content & explanations are all

Option 2: VS Code (blank file)

This involves cloning the GitHub repo and working through the exercises in VS Code. The instructions for each page (use the left sidebar to navigate to the different days). When you encounter code cells you'll copy them in and fill in the code as indicated when you get to exercises.

You can use a notebook file for this, or Python files. If you're using Python files, we recommend enabling inline [separating your code into cells](#) with the delimiter `# %%` (which effectively turns it into a notebook), or by searching for `%%` in the Code settings (which allows you to highlight & run any highlighted chunk of code using `Shift + Enter` - I've put

One of the reasons we recommend Python files over notebooks is that it's very easy for notebooks to get clogged up with code for you to return to and read later. Another is that if you work with Python files then you can import previous days of code (e.g. if you write some module in the 2nd day of material from chapter 0, and you want to use it when you get to the 3rd day of material from that chapter). Note that in this case you will need to make sure your executing code doesn't overwrite previous code. We'll show how to do this.

We recommend this option if you're more comfortable with VS Code / setting up your own environment. We strongly recommend this option to all participants of the ARENA program.

Option 3: VS Code (notebooks)

The Colab notebooks are generated directly from `ipynb` files in the ARENA repo. If you like working through notebooks but prefer VS Code to Colab (and you're comfortable setting up your own environment), you can just go through the exercises in a copy of the `_exercises.ipynb` file and look at the `_solutions.ipynb` file for reference. **It's important to make a copy** otherwise when you pull ARENA material updates you'll overwrite your filled-in exercises notebook.

ARENA structure

The ARENA repo has the following basic structure:

```
.
├── chapter0_fundamentals
│   ├── exercises
│   │   ├── part1_raytracing
│   │   │   ├── 0.1_Ray_Tracing_exercises.ipynb
│   │   │   ├── 0.1_Ray_Tracing_solutions.ipynb
│   │   │   ├── solutions.py
│   │   │   ├── tests.py
│   │   │   └── utils.py
│   │   └── part2_cnns
│   │       ├── 0.2_CNNs_exercises.ipynb
│   │       ├── 0.2_CNNs_solutions.ipynb
│   │       ├── solutions.py
│   │       ├── tests.py
│   │       └── utils.py
│   └── instructions
├── chapter1_transformer_interp
├── chapter2_rl
├── chapter3_llm_evals
└── requirements.txt
```

Each chapter contains `instructions` (which contains all the files used to generate the Streamlit pages you're working through). Each day of exercises has its own directory, which contains the exercises, the `solutions.py` file containing the solutions to the exercises, as well as sometimes `tests.py` which contains some utility functions used in that day's exercises.

How you go through the exercises depends on which option you choose:

- **Option 1: Colab** - you'll be working through a Colab notebook generated from the corresponding `_exercises.ipynb` file. You don't need to worry about the other files in the repo, because all the setup code is provided for you.
- **Option 2: VS Code (blank file)** - you should first setup your machine and clone the repo (instructions below). Then you'll work through the exercises in a blank file `answers.py` or `answers.ipynb` in the same directory as the exercises (e.g. `exercises/part1_raytracing`). You'll be copying over code blocks & filling them in when required. Run the code blocks to test your functions or display results. (You'll also learn how to run inline code in Python files).
- **Option 3: VS Code (notebooks)** - basically the same as Option 1, except you're working from the repo within VS Code. You'll need the same setup instructions as below before you start.

Again, we're highlighting an important point - **don't max and match notebook instructions and Streamlit instructions** when going through each exercise set you only need to reference the Colabs / Streamlit page / VS Code notebook. Setup code and instructions will sometimes differ slightly between the two.

If you're using option 2 or 3, you'll need to setup your workspace in VS Code - go to the "VS Code setup" section

Structure of exercises

Most of the exercises are designed to take approximately 1 day (an exception is some of the material in the second week, which is very extensive and can take much longer to complete in full). Each day is structured as follows:

- **Reading Material** (most, but not all exercises will start with reading material)
- **Setup Code** (a code block for you to run with all the relevant imports / definitions)
- **Exercises** (several sets of exercises for you to read through and complete)
- **Bonus** (most days will have suggested bonus material / paper replications, less guided than the exercises)

You should read through the reading material as indicated, run the setup code, and then work through the exercises. The exercises are generally optional, and you should think about it for a capstone project / material to return to in the future rather than a requirement.

Each day will typically be split up into subsections. We indicate the relative amount of time you should spend on each subsection in the left hand sidebar (e.g. we might have 3 sections indicated with 30% , 20% and 50% respectively). These are just guidelines - you can spend as much time as you go through the exercises.

There will usually be around 10-20 total exercises per set, spread over the different subsections. Each exercise is designed to do (usually in the form of a function with a docstring which you need to complete), as well as a test function to verify your solution. Exercises are designed to try and catch most possible ways your function can go wrong, although they won't always be able to (see later chapters). Exercises will also come with estimates for how difficult they are, and how important they are. For example, they may look like this:

Difficulty: ●●●●●
Importance: ●●●●●

You should spend up to 10-15 minutes on this exercise.

These are only rough guidelines to help you calibrate yourself as you go through the exercises, and the times will vary depending on your level of familiarity with the material. However, if you're spending a lot more than the suggested time on the exercises, it's important to **stop and move on to the next exercise**. This is really important - there's a lot of material to cover in this course and it's better to move on and not get any value from it because you've misunderstood something minor. This material has no value if you're on a question for too long - this is up to you to notice and do something about it!

Each exercise is followed by one or more code blocks, where you'll have to fill in the code as indicated. This might be a function with a docstring but whose contents is simply `raise NotImplementedError()` which you should complete. If you're in a Colab or notebook (options 1 or 3 above) you'll just be filling in those cells and running them; if you're using VS Code, you'll be copying over these code cells and filling them in before running them.

Below a code cell, you'll usually see an exercise solution in a dropdown which you can click on, like this:

► Solution

Exercises will also sometimes come with hints, in dropdowns like this one:

► Click me for a hint!

VS Code setup

This section is only relevant if you're using Option 2 or 3, i.e. if you're not using the Colab notebooks.

You can either work from your own machine if it's sufficient to run the exercises, or set up your own virtual machine. The "System Requirements" below will help you figure out if your machine meets the requirements, and the subsections will tell you whether you do / don't need to set up a VM.

System Requirements

Note - this is only relevant if you're not using Colab.

For ARENA, we've tried to make sure that the computationally intensive material can easily be scaled down to accessible as possible. If you're not using Colab, we recommend hiring a machine that at least meets the follow

- **GPU:** A GPU with at least 8GB of VRAM. >12GB would be best. You do not need lots of compute for this material any better (with the exception of transformer training day). Here's a table of budget cards available on Vast

► Comparison of various GPUs on VastAI

We recommend the **RTX 3060** or the **RTX A4000**; these will get you through the vast majority of the material in A100 if it's within your budget or you have access somehow, but for most of the content this will be overkill.

- **RAM** The usual rule of thumb is 2x your VRAM, though 16GB of RAM is probably plenty.
- **HDD** 50GB of space should be plenty, and keeping the HDD space down means the on-demand cost while y (<10USD/month).
- **CPU** Whatever comes with the GPU node is fine.
- **Bandwidth** Often forgotten, but very important. For many of the days, the material starts with downloading model (on the order of a few GBs). We recommend at least 100Mbps of bandwidth. If that's not an option, m sure you have everything downloaded ahead of time.

Local setup instructions

If your machine meets the requirements above, then you can set up a workspace as follows:

- Make & activate a virtual environment.
 - We strongly recommend using `conda` for this - you can install `conda` [here](#), and find basic instructions [here](#).
 - The command for creating a new env is `conda create --name arena-env python=3.11`.
- Activate the conda environment using `conda activate arena-env`.
- Clone the GitHub repo using `git clone https://github.com/callummcDougall/ARENA_3.0.git`.
- Install requirements using `pip install -r requirements.txt`.
 - Make sure you've navigated to the root directory of the repo before running this command.

If not, then read on to choose a cloud provider & setup a virtual machine.

► Tips for MacOS

VM setup instructions

The table below compares the 3 main providers. We generally recommend VastAI unless you're very unfamiliar better off with Lambda Labs - however if you have a cloud provider you're already familiar with then you're welcome assistance with VastAI, you can join their [Discord](#).

Feature	VastAI	RunPod	Lambda Labs
Pre-installed	Minimal setup with very few pre-installed libraries; you'll need to install most tools yourself.	Moderate setup with some pre-installed libraries.	Good setup with many pre-installed libraries.
Privacy	Not suitable for private work as machines are leased from other users.	Moderate privacy, better than VastAI but not as controlled as Lambda Labs.	Good privacy, better than VastAI and RunPod.
Cost	Cheapest option: around \$0.10-\$0.30/GB/month for idle storage; typically < \$2/day when active.	Slightly cheaper than Lambda Labs but more expensive than VastAI.	Most expensive option, but good for long-term storage.
Pausing	Easily pause instances; low idle costs if paused (but unpausing may not be immediate if machine is in use by others).	Allows pausing, but requires manual oversight to avoid costs.	Does not allow pausing.
Customizability	Highly customizable; users can select machines with specific HDD space, bandwidth, and GPU requirements.	Moderate customizability; fewer options than VastAI but better than Lambda Labs.	Low customizability.
Availability	Generally high availability with many machine options.	Better availability than Lambda Labs but fewer options than VastAI.	Low availability.

Below, we have dropdown instructions for each of the 3 providers. If you find yourself stuck on any stage, feel free to ask for help in the [Discord](#), alternatively you can just try one of the other providers.

► Setup instructions for Lambda Labs

► Setup instructions for RunPod

► Setup instructions for VastAI

Once you've connected to your instance, follow the instructions in the dropdown below (which are the same for all providers)

► Workspace setup instructions (for any provider)

Feedback

If you have any feedback on this course (e.g. bugs, confusing explanations, parts that you feel could be structured better), please let us know in the [Discord](#) [group](#), e.g. the [errata](#) channel (for mistakes) or [suggestions](#) channels (for suggestions). We're always looking for ways to improve the course and appreciate your input.

If you've spotted a bug and would like to make a PR, feel free to do so. We find it most helpful if you can edit the file that generates the Streamlit page (you can find this in e.g. [chapter0_fundamentals/instructions/pages/page.md](#)), but we also accept PRs for other files. Note that all ARENA material (markdown files, solutions file, and notebooks) are generated from the [./infrastructure/master_files](#) directory, so it's extra helpful if your PR just involves the master Python files in this directory because our devs will know how to handle PRs which edit non-master files!